



Cloud Computing Lab

Submitted To:

Sir Waqas & Sir Shoaib

Submitted By:

Hamail Fatima

Roll No:

2023-BSE-023

Section:

5(A)

Task 0 Lab Setup (Codespace & GH CLI)

All actions below should be executed inside the Codespace shell.
Create Codespace & connect:

create or open codespace via GH CLI (example)

```
gh repo create CC_<YourName>_<YourRollNumber>/Lab11 --public
gh codespace create --repo <user_name>/Lab11
gh codespace list
gh codespace ssh -c <your_codespace_name>
```

- Save screenshot as: taskA_codespace_create_and_list.png – output showing repo creation/codespace list.

```
PS C:\Users\admin\CC_hamail-maker_023_Lab11> gh codespace create --repo Hamail-maker/CC_hamail-maker_023_Lab11
✓ Codespaces usage for this repository is paid for by Hamail-maker
? Choose Machine Type: 2 cores, 8 GB RAM, 32 GB storage
reimagined-orbit-x54jpg6pjgg5hvj9p
PS C:\Users\admin\CC_hamail-maker_023_Lab11> gh codespace list
```

NAME	DISPLAY NAME	REPOSITORY	BRANCH	STATE	CREATED AT
potential-eureka-q7xw45r4wg5...	potential eureka	Hamail-maker/my-lab-repo	main*	Shutdown	about 4 days ago
reimagined-orbit-x54jpg6pjgg...	reimagined orbit	Hamail-maker/CC_hamail-...	main	Available	less than a minute ago

```
PS C:\Users\admin\CC_hamail-maker_023_Lab11> |
```

- Save screenshot as: taskA_codespace_ssh_connected.png – terminal inside the Codespace shell after ssh.

```
PS C:\Users\admin\CC_hamail-maker_023_Lab11> gh codespace ssh -c reimagined-orbit-x54jpg6pjgg5hvj9p
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $
```

Task 1 — Provider & Basic variable (variable precedence)

1. In Codespace create main.tf:

touch main.tf

- Save screenshot as: task1_touch_main_tf.png — terminal showing touch main.tf.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ touch main.tf
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

2. Edit main.tf and add provider:

```
provider "aws" {
  shared_config_files    = ["~/aws/config"]
  shared_credentials_files = ["~/aws/credentials"]
}
```

- Save screenshot as: task1_main_tf_provider.png — editor showing provider block saved in main.tf.

```
GNU nano 7.2 main.tf *
```

```
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}
```

3. Initialize:

terraform init

- **Save screenshot as:** task1_terraform_init.png — terraform init output.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform init  
Initializing the backend...  
Initializing provider plugins...  
- Finding latest version of hashicorp/aws...  
- Installing hashicorp/aws v6.27.0...  
- Installed hashicorp/aws v6.27.0 (signed by HashiCorp)  
Terraform has created a lock file .terraform.lock.hcl to record the provider  
selections it made above. Include this file in your version control repository  
so that Terraform can guarantee to make the same selections by default when  
you run "terraform init" in the future.  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan" to see  
any changes that are required for your infrastructure. All Terraform commands  
should now work.  
  
If you ever set or change modules or backend configuration for Terraform,  
rerun this command to reinitialize your working directory. If you forget, other  
commands will detect it and remind you to do so if necessary.  
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

4. Define a variable and output:

```
variable "subnet_cidr_block" {  
  type = string  
}
```

```
output "subnet_cidr_block_output" {  
  value = var.subnet_cidr_block  
}
```

- **Save screenshot as:** task1_variable_and_output_added.png — main.tf showing variable and output blocks.

```
GNU nano 7.2 main.tf *
```

```
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
variable "subnet_cidr_block" {  
  type = string  
}  
  
output "subnet_cidr_block_output" {  
  value = var.subnet_cidr_block  
}
```

5. Run apply (first time without defaults):

terraform apply -auto-approve

it will prompt for the value of subnet_cidr_block

- **Save screenshot as:** task1_apply_prompt_for_var.png — terminal showing terraform prompting for variable input.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve
var.subnet_cidr_block
  Enter a value: 10.0.10.0/24

Changes to Outputs:
  + subnet_cidr_block_output = "10.0.10.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
subnet_cidr_block_output = "10.0.10.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

6. Add a default to the variable:

```
variable "subnet_cidr_block" {
  type    = string
  default = "10.0.0.0/24"
}
```

Run:

terraform apply -auto-approve

will use default without prompting

- **Save screenshot as:** task1_apply_with_default.png — apply output showing default used.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.10.0/24" -> "10.0.0.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
subnet_cidr_block_output = "10.0.0.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

7. Export environment variable in Codespace shell:

export TF_VAR_subnet_cidr_block=10.0.20.0/24

terraform apply -auto-approve

output should show environment value

- **Save screenshot as:** task1_env_var_set_and_apply.png — terminal showing export and apply using TF_VAR value.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ export TF_VAR_subnet_cidr_block=10.0.20.0/24
terraform apply -auto-approve

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.0.0/24" -> "10.0.20.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
subnet_cidr_block_output = "10.0.20.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

8. Create terraform.tfvars overriding values:

touch terraform.tfvars

inside terraform.tfvars:

subnet_cidr_block = "10.0.30.0/24"

terraform apply -auto-approve

terraform.tfvars has priority over default and env

- **Save screenshot as:** task1_terraform_tfvars_and_apply.png — cat terraform.tfvars and apply output showing its value used.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.20.0/24" -> "10.0.30.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.30.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

9. Override with -var:

terraform apply -auto-approve -var "subnet_cidr_block=10.0.40.0/24"

-var is highest precedence

- **Save screenshot as:** task1_var_override_with_dash_var.png — command and apply showing -var value used.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve -var "subnet_cidr_block=10.0.40.0/24"

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.30.0/24" -> "10.0.40.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.40.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

10. Show and unset env var:

printenv | grep TF_VAR_

unset TF_VAR_subnet_cidr_block

printenv | grep TF_VAR_

- **Save screenshot as:** task1_printenv_tf_var_and_unset.png — showing TF_VAR present and then removed.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ printenv | grep TF_VAR_
unset TF_VAR_subnet_cidr_block
printenv | grep TF_VAR_
TF_VAR_subnet_cidr_block=10.0.20.0/24
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

Task 2 — Variable validation & sensitive / ephemeral variables

1. Replace subnet_cidr_block variable with this (validation included):

```
variable "subnet_cidr_block" {
  type      = string
  default   = ""
  description = "CIDR block to assign to the application subnet"
  sensitive = false
  nullable  = false
  ephemeral = false
```

```

validation {
  condition = can(regex("^[0-9]{1,3}\\.{3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
  error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24."
}
}

```

- **Save screenshot as:** task2_subnet_variable_with_validation.png — main.tf showing validation block.

```

GNU nano 7.2 main.tf *

provider "aws" {
  shared_config_files = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}

variable "subnet_cidr_block" {
  type = string
  default = ""
  description = "CIDR block to assign to the application subnet"
  sensitive = false
  nullable = false
  ephemeral = false

  validation {
    condition = can(regex("^[0-9]{1,3}\\.{3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
    error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24."
  }
}

```

2. Test validation failure:
 terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"
 # should show validation error

- **Save screenshot as:** task2_subnet_validation_error.png — terminal showing validation error message.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"

Error: Invalid value for variable

on main.tf line 11:
11: variable "subnet_cidr_block" {
    |   var.subnet_cidr_block is "10.0.0"

The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24.

This was checked by the validation rule at main.tf:19,3-13.

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $

```

3. Create a sensitive variable api_session_token and output (sensitive):
 variable "api_session_token" {
 type = string
 default = ""
 description = "Short-lived API session token used during apply operations"
 sensitive = true
 nullable = false
 ephemeral = false

 validation {
 condition = can(regex("^[A-Za-z0-9-]{20}\$", var.api_session_token))

```

    error_message = "The API session token must be at least 20 characters and contain only
letters, numbers, hyphens, or underscores."
}
}

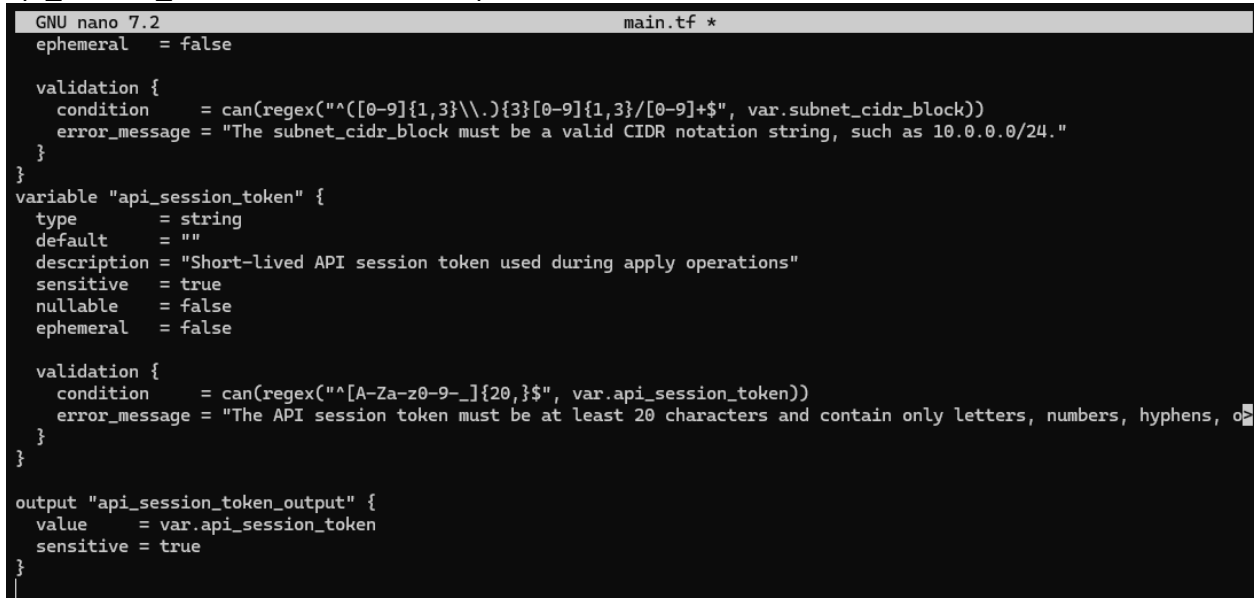
```

```

output "api_session_token_output" {
  value    = var.api_session_token
  sensitive = true
}

```

- **Save screenshot as:** task2_api_token_variable_added.png — main.tf showing api_session_token and sensitive output.



```

GNU nano 7.2                                main.tf *
ephemeral    = false

validation {
  condition   = can(regex("^[0-9]{1,3}\\.{3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
  error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24."
}
}

variable "api_session_token" {
  type        = string
  default     = ""
  description = "Short-lived API session token used during apply operations"
  sensitive   = true
  nullable    = false
  ephemeral   = false

  validation {
    condition   = can(regex("^[A-Za-z0-9-]{20,}$", var.api_session_token))
    error_message = "The API session token must be at least 20 characters and contain only letters, numbers, hyphens, o
  }
}

output "api_session_token_output" {
  value    = var.api_session_token
  sensitive = true
}

```

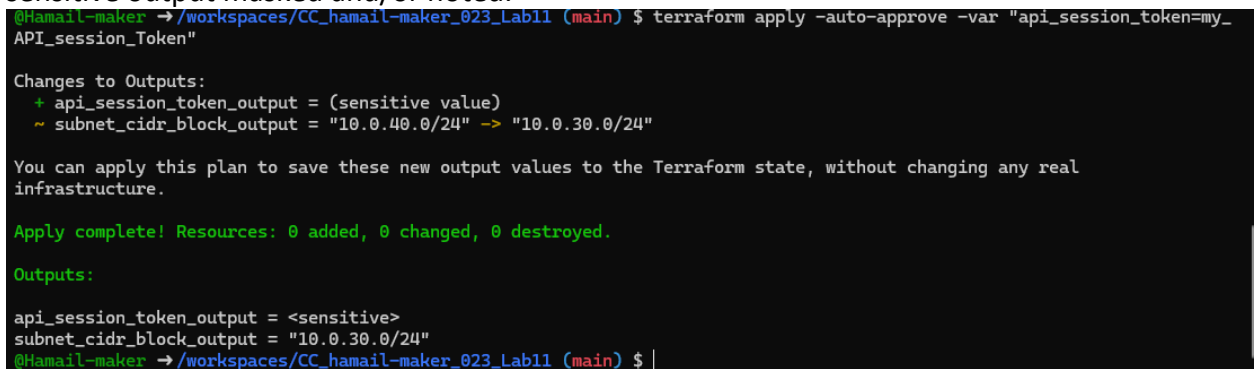
4. Run with -var to observe sensitive output behavior:

```

terraform apply -auto-approve -var "api_session_token=my_API_session_Token"
# output will be marked sensitive; check terraform.tfstate for outputs

```

- **Save screenshot as:** task2_api_token_apply_sensitive.png — apply output showing sensitive output masked and/or noted.



```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve -var "api_session_token=my_
API_session_Token"

Changes to Outputs:
+ api_session_token_output = (sensitive value)
~ subnet_cidr_block_output = "10.0.40.0/24" -> "10.0.30.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.30.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```

5. Check terraform.state for the sensitive output:

- You should find an outputs section similar to:

```

"api_session_token_output": {
  "value": "my_API_session_Token",
  "type": "string",
  "sensitive": true
}

```

- **Save screenshot as:** task2_check_terraform_state_api_token.png — showing terraform.tfstate outputs containing "api_session_token_output" with sensitive = true.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ cat terraform.tfstate | grep -A 5 api_session_token_output
t
  "api_session_token_output": {
    "value": "my_API_session_Token",
    "type": "string",
    "sensitive": true
  },
  "subnet_cidr_block_output": {
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

6. Make variable ephemeral to hide from state:

```
variable "api_session_token" {
  ...
  ephemeral = true
  ...
}
```

- **Save screenshot as:** task2_api_token_ephemeral_error.png — terminal showing error or behavior when trying to output ephemeral variable.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve -var "api_session_token=my_API_session_Token"

Error: Duplicate variable declaration

   on main.tf line 42:
  42: variable "api_session_token" {

A variable named "api_session_token" was already declared at main.tf:24,1-29. Variable names must be unique within a module.

Error: Argument or block definition required

   on main.tf line 43, in variable "api_session_token":
  43:   ...

An argument or block definition is required here.

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

7. Set default to test local default:

```
variable "api_session_token" {
  default = "my_API_session_Token"
  sensitive = true
  ephemeral = false
  ...
}
```

terraform apply -auto-approve

works and stored in state (but output remains sensitive)

- **Save screenshot as:** task2_api_token_default_apply.png — apply output showing default sensitive value accepted.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

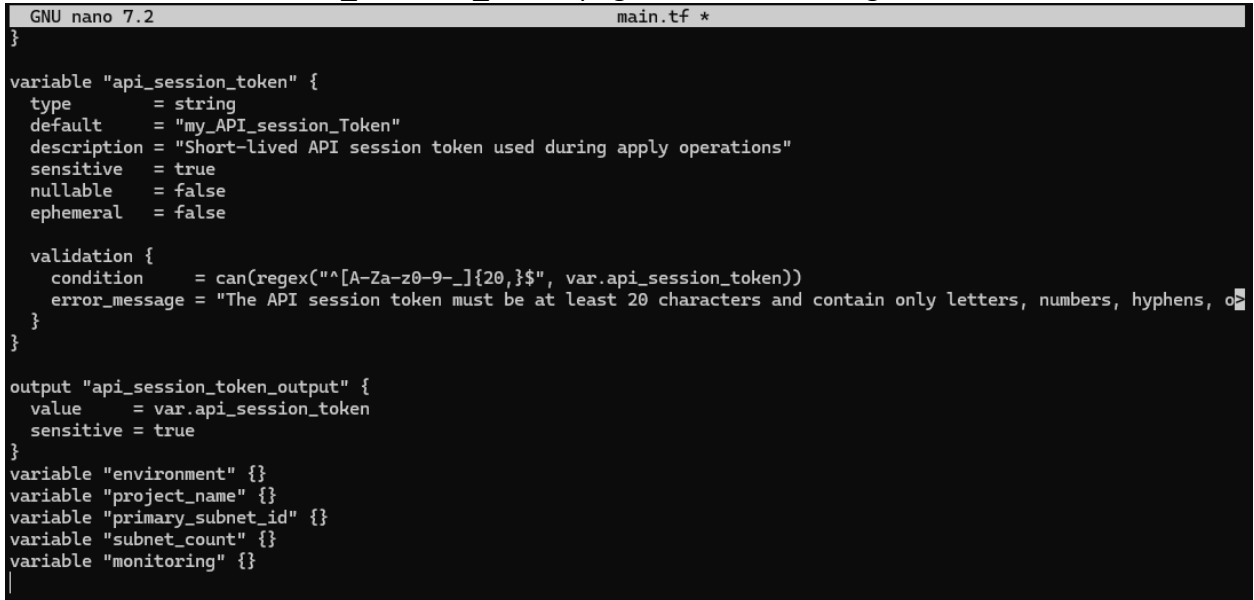
api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.30.0/24"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

Task 3 — Project-level variables, locals, and outputs

1. Add variables to main.tf:

```
variable "environment" {}
variable "project_name" {}
variable "primary_subnet_id" {}
variable "subnet_count" {}
variable "monitoring" {}
```

- **Save screenshot as:** task3_variables_added.png — main.tf showing new variables.



```
GNU nano 7.2 main.tf *
}

variable "api_session_token" {
  type        = string
  default     = "my_API_session_Token"
  description = "Short-lived API session token used during apply operations"
  sensitive   = true
  nullable    = false
  ephemeral   = false

  validation {
    condition     = can(regex("^[A-Za-z0-9-]{20,}$", var.api_session_token))
    error_message = "The API session token must be at least 20 characters and contain only letters, numbers, hyphens, o"
  }
}

output "api_session_token_output" {
  value     = var.api_session_token
  sensitive = true
}

variable "environment" {}
variable "project_name" {}
variable "primary_subnet_id" {}
variable "subnet_count" {}
variable "monitoring" {}
|
```

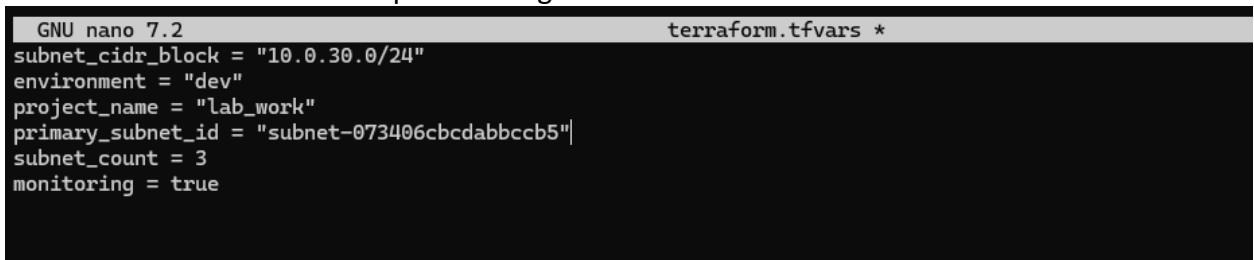
2. Populate terraform.tfvars *after* discovering actual subnet id for availability zone me-central-1a:

```
aws ec2 describe-subnets \
  --filters "Name=availability-zone,Values=me-central-1a" \
  --query "Subnets[].SubnetId" \
  --output text
```

Set values in terraform.tfvars:

```
environment = "dev"
project_name = "lab_work"
primary_subnet_id = "<subnet-id-of-me-central-1a>"
subnet_count = 3
monitoring = true
```

- **Save screenshot as:** task3_terraform_tfvars_populated.png — terraform.tfvars content and aws describe-subnets output showing subnet id.



```
GNU nano 7.2 terraform.tfvars *
subnet_cidr_block = "10.0.30.0/24"
environment = "dev"
project_name = "lab_work"
primary_subnet_id = "subnet-073406cbcdabbccb5"
subnet_count = 3
monitoring = true
```

3. Create locals.tf with:

```
locals {
  resource_name = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
```

```

subnet_count      = var.subnet_count
is_production      = var.environment == "prod"
monitoring_enabled = var.monitoring || local.is_production
}

```

- **Save screenshot as:** task3_locals_tf_created.png — locals.tf content saved.

```

GNU nano 7.2                                locals.tf *
locals {
  resource_name      = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count       = var.subnet_count
  is_production       = var.environment == "prod"
  monitoring_enabled  = var.monitoring || local.is_production
}
|

```

4. Add outputs to main.tf:

```

output "resource_name" {
  value = local.resource_name
}
output "primary_public_subnet" {
  value = local.primary_public_subnet
}
output "subnet_count" {
  value = local.subnet_count
}
output "is_production" {
  value = local.is_production
}
output "monitoring_enabled" {
  value = local.monitoring_enabled
}

```

- Run:

```

terraform apply -auto-approve
# will show all the output values

```

- **Save screenshot as:** task3_outputs_apply.png — terraform apply output showing these outputs.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve

Changes to Outputs:
+ is_production           = false
+ monitoring_enabled      = true
+ primary_public_subnet   = "subnet-073406cbcdabbccb5"
+ resource_name           = "lab_work-dev"
+ subnet_count            = 3

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
api_session_token_output = <sensitive>
is_production             = false
monitoring_enabled        = true
primary_public_subnet     = "subnet-073406cbcdabbccb5"
resource_name             = "lab_work-dev"
subnet_cidr_block_output  = "10.0.30.0/24"
subnet_count              = 3
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```

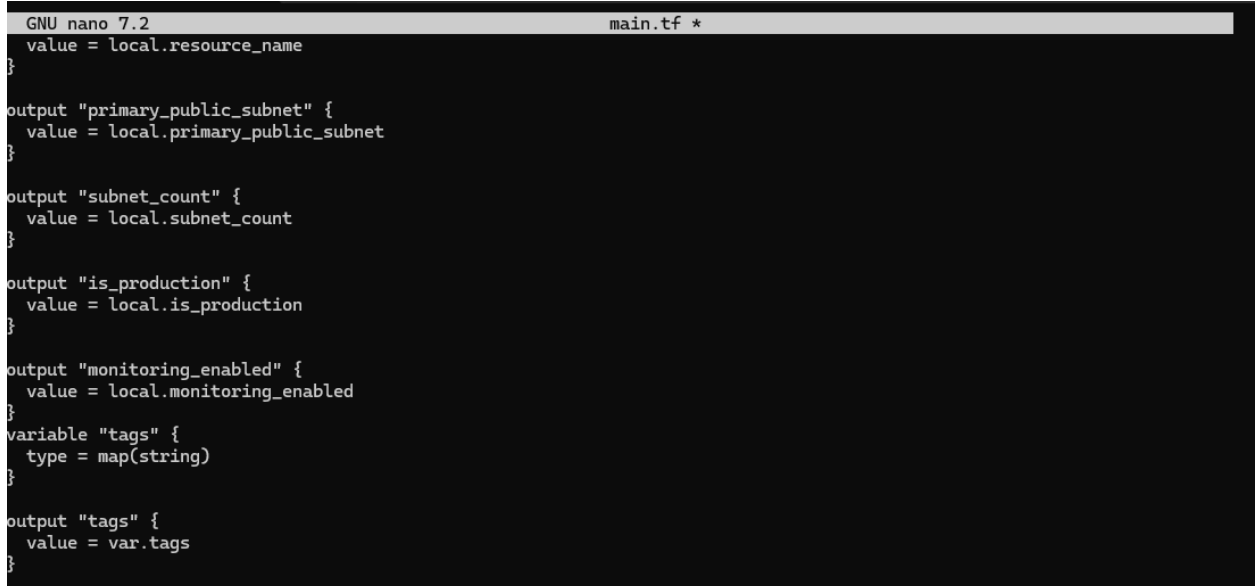
Task 4 — Maps and Objects

1. Map variable in main.tf:

```
variable "tags" {  
  type = map(string)  
}
```

```
output "tags" {  
  value = var.tags  
}
```

- **Save screenshot as:** task4_tags_variable_added.png — main.tf showing tags variable and output.



```
GNU nano 7.2 main.tf *  
value = local.resource_name  
}  
  
output "primary_public_subnet" {  
  value = local.primary_public_subnet  
}  
  
output "subnet_count" {  
  value = local.subnet_count  
}  
  
output "is_production" {  
  value = local.is_production  
}  
  
output "monitoring_enabled" {  
  value = local.monitoring_enabled  
}  
variable "tags" {  
  type = map(string)  
}  
  
output "tags" {  
  value = var.tags  
}
```

2. In terraform.tfvars:

```
tags = {  
  Environment = "dev"  
  Project     = "sample-app"  
  Owner       = "platform-team"  
}
```

Run:

```
terraform apply -auto-approve  
# check tags output
```

- **Save screenshot as:** task4_tags_output.png — apply output showing tags returned.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve

Changes to Outputs:
+ tags = {
+   Environment = "dev"
+   Owner       = "platform-team"
+   Project     = "sample-app"
+ }

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
is_production             = false
monitoring_enabled        = true
primary_public_subnet     = "subnet-073406cbcdabbccb5"
resource_name             = "lab_work-dev"
subnet_cidr_block_output  = "10.0.30.0/24"
subnet_count              = 3
tags = tomap({
  "Environment" = "dev"
  "Owner"       = "platform-team"
  "Project"     = "sample-app"
})
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ s|
```

3. Define object variable:

```
variable "server_config" {
  type = object({
    name          = string
    instance_type = string
    monitoring     = bool
    storage_gb     = number
    backup_enabled = bool
  })
}
```

```
output "server_config" {
  value = var.server_config
}
```

In terraform.tfvars:

```
server_config = {
  name          = "web-server"
  instance_type = "t3.micro"
  monitoring     = true
  storage_gb     = 20
  backup_enabled = false
}
```

- **Save screenshot as:** task4_server_config_output.png — apply output showing server_config object.

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
api_session_token_output = <sensitive>
is_production = false
monitoring_enabled = true
primary_public_subnet = "subnet-073406cbcdabbccb5"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) \$ |

Task 5 — Collections: list, tuple, set & mutation via locals

In this task you will define collection variables (list, tuple, set), observe their behavior, then perform mutations via locals and compare results.

Feature	List	Tuple	Set
Order preserved	✓	✓	✗
Allows duplicates	✓	✓	✗
Mixed types	✗	✓	✗
Fixed size	✗	✓	✗
Mutable	✓	✗	✓
Best for	Flexible sequences	Structured records	Unique collections

1. In main.tf define:

```
variable "server_names" {
  type = list(string)
  default = ["web-2", "web-1", "web-2"]
}

variable "server_metadata" {
  type = tuple([string, number, bool])
  default = ["web-1", 4, true]
}

variable "availability_zones" {
  type = set(string)
```

```

    default = ["me-central-1b", "me-central-1a", "me-central-1b"]
}

```

```

output "compare_collections" {
  value = {
    list_example = var.server_names
    tuple_example = var.server_metadata
    set_example = var.availability_zones
  }
}

```

- **Save screenshot as:** task5_collections_defined.png — main.tf showing collection variables and output.

```

GNU nano 7.2                                main.tf *
output "server_config" {
  value = var.server_config
}
variable "server_names" {
  type    = list(string)
  default = ["web-2", "web-1", "web-2"]
}
variable "server_metadata" {
  type    = tuple([string, number, bool])
  default = ["web-1", 4, true]
}
variable "availability_zones" {
  type    = set(string)
  default = ["me-central-1b", "me-central-1a", "me-central-1b"]
}
output "compare_collections" {
  value = {
    list_example = var.server_names
    tuple_example = var.server_metadata
    set_example = var.availability_zones
  }
}

```

2. Run:

terraform apply -auto-approve

Observe the output — note ordering, duplicates, and that sets are displayed without duplicates and order is not guaranteed.

- **Save screenshot as:** task5_compare_collections.png — apply output comparing of collections.

```

Outputs:
api_session_token_output = <sensitive>
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "me-central-1a",
    "me-central-1b",
  ])
  "tuple_example" = [
    "web-1",
    4,
    true,
  ]
}
is_production = false
monitoring_enabled = true
primary_public_subnet = "subnet-073406cbcdabbccb5"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "tags" = {}
}

```

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve
```

Changes to Outputs:

```
+ compare_collections = {
+   list_example = [
+     "web-2",
+     "web-1",
+     "web-2",
+   ]
+   set_example = [
+     "me-central-1a",
+     "me-central-1b",
+   ]
+   tuple_example = [
+     "web-1",
+     4,
+     true,
+   ]
+ }
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

3. In locals.tf add mutations: Create or edit locals.tf and add the following locals to demonstrate mutation behavior. Note: tuples are immutable in Terraform's type system, but many operations convert them to lists for evaluation.

```
locals {
  mutated_list = setunion(var.server_names, ["web-3"])
  mutated_tuple = setunion(var.server_metadata, ["web-2"])
  mutated_set = setunion(var.availability_zones, ["me-central-1c"])
}
```

- **Save screenshot as:** task5_locals_mutations.png — locals.tf showing mutated values.

```
GNU nano 7.2                                locals.tf *
locals {
  resource_name      = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count       = var.subnet_count
  is_production       = var.environment == "prod"
  monitoring_enabled  = var.monitoring || local.is_production
}
locals {
  mutated_list = setunion(var.server_names, ["web-3"])
  mutated_tuple = setunion(var.server_metadata, ["web-2"])
  mutated_set = setunion(var.availability_zones, ["me-central-1c"])
}
```

4. Add comparison output in main.tf:
output "mutation_comparison" {
 value = {
 original_tuple = var.server_metadata
 mutated_tuple = local.mutated_tuple
 }
}

Run:

```
terraform apply -auto-approve
```

Observe the difference between the original tuple and the mutated result. Note that Terraform may represent the mutated tuple as a list and the ordering/duplicates behavior will reflect setunion semantics.

- **Save screenshot as:** task5_mutation_comparison.png — apply output comparing original and mutated tuple.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
api_session_token_output = <sensitive>
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "me-central-1a",
    "me-central-1b",
  ])
  "tuple_example" = [
    "web-1",
    4,
    true,
  ]
}
is_production = false
monitoring_enabled = true
mutation_comparison = {
  "mutated_tuple" = toset([
    "4",
    "true",
    "web-1",
  ])
  "original_tuple" = [
    "web-1",
    4,
    true,
  ]
}
primary_public_subnet = "subnet-073406cbcdabbccb5"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
```

Task 6 — Null, any type & dynamic values

1. Null variable:

```
variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default   = null
}
```

- **Save screenshot as:** task6_optional_tag_variable.png — main.tf showing optional_tag variable.

```

GNU nano 7.2                                     main.tf *
}

variable "availability_zones" {
  type      = set(string)
  default = ["me-central-1b", "me-central-1a", "me-central-1b"]
}

output "compare_collections" {
  value = {
    list_example  = var.server_names
    tuple_example = var.server_metadata
    set_example   = var.availability_zones
  }
}

output "mutation_comparison" {
  value = {
    original_tuple = var.server_metadata
    mutated_tuple  = local.mutated_tuple
  }
}

variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default   = null
}

```

2. Merge tags in locals.tf:

```

locals {
  server_tags = merge(
    { Name = "web-server" },
    var.optional_tag != null ? { Custom = var.optional_tag } : {}
  )
}

```

- **Save screenshot as:** task6_locals_merge.png — locals.tf showing merge logic.

```

GNU nano 7.2                                     locals.tf *
locals {
  resource_name      = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count       = var.subnet_count
  is_production       = var.environment == "prod"
  monitoring_enabled  = var.monitoring || local.is_production
}

locals {
  mutated_list = setunion(var.server_names, ["web-3"])
  mutated_tuple = setunion(var.server_metadata, ["web-2"])
  mutated_set = setunion(var.availability_zones, ["me-central-1c"])
}

locals {
  server_tags = merge(
    { Name = "web-server" },
    var.optional_tag != null ? { Custom = var.optional_tag } : {}
  )
}

```

3. Output:

```

output "optional_tag" {
  value = local.server_tags
}

```

Run:

terraform apply -auto-approve

- **Save screenshot as:** task6_optional_tag_no_value.png — apply output with no optional_tag

set.

```
optional_tag = {  
  "Name" = "web-server"
```

4. Add in terraform.tfvars:

```
optional_tag = "dev"
```

```
terraform apply -auto-approve
```

```
# observe that Custom tag appears
```

- **Save screenshot as:** task6_optional_tag_with_value.png — apply output after setting optional_tag in terraform.tfvars.

```
}  
optional_tag = {  
  "Custom" = "dev"  
  "Name" = "web-server"  
}
```

5. Any type variable:

```
variable "dynamic_value" {
```

```
  type    = any
```

```
  description = "A variable that can accept any data type"
```

```
  default    = null
```

```
}
```

```
output "value_received" {
```

```
  value = var.dynamic_value
```

```
}
```

6. Now test the dynamic (any) variable with different types. For each change, update terraform.tfvars with the specified value and run terraform apply -auto-approve, then capture the output.

a) String

- In terraform.tfvars:

```
dynamic_value = "hello"
```

- Run:

```
terraform apply -auto-approve
```

- **Save screenshot as:** task6_dynamic_value_string.png — apply output when dynamic_value = "hello".

```
subnet_cidr_block_output = "10.0.30.0/24"
```

```
subnet_count = 3
```

```
tags = tomap({
```

```
  "Environment" = "dev"
```

```
  "Owner" = "platform-team"
```

```
  "Project" = "sample-app"
```

```
})
```

```
value_received = "hello"
```

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

b) Number

- Change terraform.tfvars:

```
dynamic_value = 42
```

- Run:

```
terraform apply -auto-approve
```

- **Save screenshot as:** task6_dynamic_value_number.png — apply output when dynamic_value = 42.

```
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = 42
```

c) List

- Change terraform.tfvars:
dynamic_value = ["a", "b", "c"]
- Run:
terraform apply -auto-approve
- **Save screenshot as:** task6_dynamic_value_list.png — apply output when dynamic_value is a list.

```
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = [
  "a",
  "b",
  "c",
]
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

d) Map / Object

- Change terraform.tfvars:
dynamic_value = {
 name = "server"
 cpu = 4
}
- Run:
terraform apply -auto-approve
- **Save screenshot as:** task6_dynamic_value_map.png — apply output when dynamic_value is a map/object.

```
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = {
  "cpu" = 4
  "name" = "server"
}
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

e) Null

- Change terraform.tfvars.

- dynamic_value = null
- Run:
 - terraform apply -auto-approve
- Save screenshot as:** task6_dynamic_value_null.png — apply output when dynamic_value = null.

```
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

Task 7 — Git ignore

Create .gitignore:

touch .gitignore

Add entries:

.terraform/*

*.tfstate

.tfstate.

*.tfvars

*.pem

- Save screenshot as:** task7_gitignore_created.png — .gitignore file content showing entries.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ touch .gitignore
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ nano .gitignore
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ cat .gitignore
.terraform/*
*.tfstate
*.tfstate.*
*.tfvars
*.pem
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

Task 8 — Clean-up then build real infra (VPC, Subnet, IGW, routing, default route table)

In this task you will clean previous example values and build a simple VPC + Subnet, attach an Internet Gateway, and configure routing (first with a custom route table and association, then switch to the default route table).

Perform all commands inside your Codespace shell.

- Clean previous files
 - Remove all variable assignments from terraform.tfvars (empty the file or delete it).
 - Remove all content from locals.tf (delete or empty the file).
 - Replace main.tf contents with only the provider block below (start fresh).

Provider block to put in main.tf:

```
provider "aws" {
  shared_config_files  = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
```

- Save screenshot as:** task8_clean_files.png — showing cleaned terraform.tfvars, locals.tf, and main.tf.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ > terraform.tfvars
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ > locals.tf
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ nano main.tf
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```

2. Define variables in main.tf Add these variable declarations to main.tf (below the provider block):

```

variable "vpc_cidr_block" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}

```

- **Save screenshot as:** task8_variables_recreated.png — main.tf showing the four variables added.

```

GNU nano 7.2 main.tf *
provider "aws" {
  shared_config_files      = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
variable "vpc_cidr_block" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}

```

3. Create VPC in main.tf Add the VPC resource to main.tf:

```

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

```

- **Save screenshot as:** task8_vpc_resources_added.png — main.tf showing aws_vpc resource (after adding VPC).

```

resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}
|

```

4. Create Subnet in the VPC Add the subnet resource to main.tf:

```

resource "aws_subnet" "myapp_subnet_1" {
  vpc_id      = aws_vpc.myapp_vpc.id
  cidr_block   = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

```

- **Save screenshot as:** task8_subnet_resources_added.png — main.tf showing aws_subnet resource (after adding subnet).

```
resource "aws_subnet" "myapp_subnet_1" {
  vpc_id          = aws_vpc.myapp_vpc.id
  cidr_block      = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}
```

5. Populate terraform.tfvars In terraform.tfvars add:

```
vpc_cidr_block    = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix        = "dev"
```

- **Save screenshot as:** task8_terraform_tfvars_vpc_values.png — terraform.tfvars content showing the values added.

```
GNU nano 7.2                                     terraform.tfvars *
vpc_cidr_block    = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix        = "dev"
|
```

6. Apply to create VPC and Subnet Initialize (if needed) and apply:

terraform init

terraform apply -auto-approve

Verify in AWS Console that the VPC and Subnet were created.

- **Save screenshot as:** task8_vpc_subnet_apply.png — terraform apply output showing VPC and subnet creation on AWS console screenshot.

```
aws_vpc.myapp_vpc: Creating...
aws_vpc.myapp_vpc: Creation complete after 2s [id=vpc-01fbe6b5b3c6ba1c3]
aws_subnet.myapp_subnet_1: Creating...
aws_subnet.myapp_subnet_1: Creation complete after 1s [id=subnet-08c0bf6ef5f2171b5]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

7. Create Internet Gateway and Route Table (custom) Add the Internet Gateway and a custom Route Table to main.tf:

```
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}
```

```
resource "aws_route_table" "myapp_route_table" {
  vpc_id = aws_vpc.myapp_vpc.id
```

```
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }
}
```

```

}

tags = {
  Name = "${var.env_prefix}-rt"
}
}

```

- **Save screenshot as:** task8_igw_route_table_before_apply.png — AWS console showing IGW and route table resources (before apply).

```

resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_route_table" "myapp_route_table" {
  vpc_id = aws_vpc.myapp_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

Apply:

terraform apply -auto-approve

Verify IGW and route table in AWS Console.

- **Save screenshot as:** task8_igw_route_table_after_apply.png — terraform apply output showing IGW and route table creation on AWS console.

```

Plan: 2 to add, 0 to change, 0 to destroy.
aws_internet_gateway.myapp_igw: Creating...
aws_internet_gateway.myapp_igw: Creation complete after 1s [id=igw-0d34b6523d938f67f]
aws_route_table.myapp_route_table: Creating...
aws_route_table.myapp_route_table: Creation complete after 1s [id=rtb-07a4bbd40c7b410af]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```

8. Associate the Route Table with the Subnet Add the association resource to main.tf:

```

resource "aws_route_table_association" "a_rtb_subnet" {
  subnet_id    = aws_subnet.myapp_subnet_1.id
  route_table_id = aws_route_table.myapp_route_table.id
}

```

Apply:

terraform apply -auto-approve

Verify association in AWS Console (Subnet → Route Table).

- **Save screenshot as:** task8_association_apply.png — terraform apply output showing association creation.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-01fbc6b5b3c6ba1c3]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0d34b6523d938f67f]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-08c0bf6ef5f2171b5]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-07a4bbd40c7b410af]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
+ create

Terraform will perform the following actions:

# aws_route_table_association.a_rtb_subnet will be created
+ resource "aws_route_table_association" "a_rtb_subnet" {
  + id           = (known after apply)
  + region      = "me-central-1"
  + route_table_id = "rtb-07a4bbd40c7b410af"
  + subnet_id    = "subnet-08c0bf6ef5f2171b5"
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.a_rtb_subnet: Creating...
aws_route_table_association.a_rtb_subnet: Creation complete after 1s [id=rtbassoc-0aacf0c8552815c21]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

```

9. Switch to default route table (use VPC default route table) Now remove (or comment out) the custom route table and association resources from main.tf:

- Remove the aws_route_table "myapp_route_table" block.
- Remove the aws_route_table_association "a_rtb_subnet" block.

Then add this resource to update the default route table with a route to the IGW:

```

resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

```

```

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

```

```

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

- **Save screenshot as:** task8_default_route_table.png — main.tf showing aws_default_route_table resource (after change).

```

route {
  cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.myapp_igw.id
}

tags = {
  Name = "${var.env_prefix}-rt"
}
}
resource "aws_default_route_table" "main_rt" {
  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

```

Apply:

terraform apply -auto-approve

Verify in AWS Console that the VPC's default route table has the 0.0.0.0/0 → IGW route and that the custom route table (if previously created) is no longer in use.

- **Save screenshot as:** task8_default_route_table_apply.png — terraform apply output showing default route table update.

```

] -> null
- tags          = {
  - "Name" = "dev-rt"
} -> null
- tags_all      = {
  - "Name" = "dev-rt"
} -> null
- vpc_id        = "vpc-01fbe6b5b3c6ba1c3" -> null
}

# aws_route_table_association.a_rtb_subnet will be destroyed
# (because aws_route_table_association.a_rtb_subnet is not in configuration)
- resource "aws_route_table_association" "a_rtb_subnet" {
  - id          = "rtbassoc-0aacf0c8552815c21" -> null
  - region      = "me-central-1" -> null
  - route_table_id = "rtb-07a4bbd40c7b410af" -> null
  - subnet_id    = "subnet-08c0bf6ef5f2171b5" -> null
  # (1 unchanged attribute hidden)
}

Plan: 1 to add, 0 to change, 2 to destroy.
aws_route_table_association.a_rtb_subnet: Destroying... [id=rtbassoc-0aacf0c8552815c21]
aws_route.default_rt_route: Creating...
aws_route_table_association.a_rtb_subnet: Destruction complete after 0s
aws_route_table.myapp_route_table: Destroying... [id=rtb-07a4bbd40c7b410af]
aws_route.default_rt_route: Creation complete after 0s [id=r-rtb-0cf10eacea458751b1080289494]
aws_route_table.myapp_route_table: Destruction complete after 1s

Apply complete! Resources: 1 added, 0 changed, 2 destroyed.

```

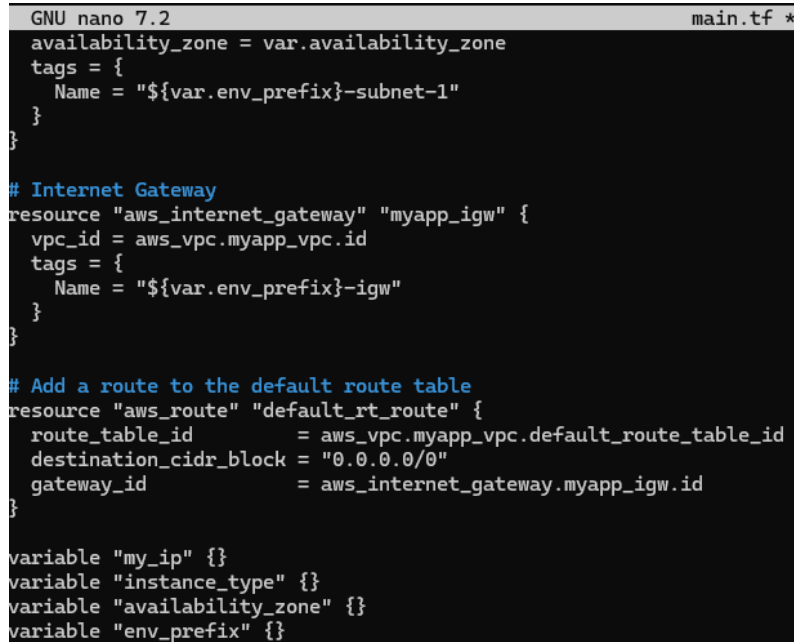
Task 9 — Security Group, Key Pair, EC2 Instance, user_data & nginx

This task walks you through creating a security group, creating an EC2 key pair, launching an EC2 instance, verifying SSH access, and installing nginx via user_data (inline and from a script). Perform all commands from your Codespace shell.

1. Add variables to main.tf Add these variables to your main.tf:

```
variable "my_ip" {}
```

- **Save screenshot as:** task9_my_ip_variable_added.png — main.tf showing my_ip and other variables added.



```
GNU nano 7.2 main.tf *
availability_zone = var.availability_zone
tags = {
  Name = "${var.env_prefix}-subnet-1"
}
}

# Internet Gateway
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

# Add a route to the default route table
resource "aws_route" "default_rt_route" {
  route_table_id      = aws_vpc.myapp_vpc.default_route_table_id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id          = aws_internet_gateway.myapp_igw.id
}

variable "my_ip" {}
variable "instance_type" {}
variable "availability_zone" {}
variable "env_prefix" {}
```

2. Get your public IP and set terraform.tfvars From the Codespace shell:

```
curl icanhazip.com
```

Copy the IP and add to terraform.tfvars:

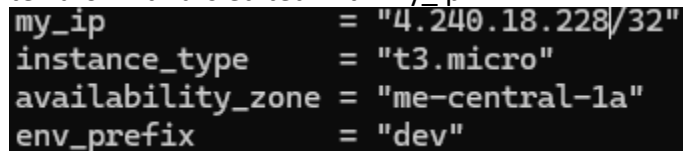
```
my_ip = "<your_ip>/32"
```

```
instance_type = "t3.micro"
```

```
availability_zone = "me-central-1a" # or your chosen AZ
```

```
env_prefix = "dev"
```

- **Save screenshot as:** task9_public_ip_curl.png — terminal showing curl icanhazip.com and terraform.tfvars edited with my_ip.



```
my_ip      = "4.240.18.228/32"
instance_type = "t3.micro"
availability_zone = "me-central-1a"
env_prefix  = "dev"
```

3. Create the Security Group (main.tf) Add this resource to main.tf:

```
resource "aws_default_security_group" "myapp_sg" {
  vpc_id = aws_vpc.myapp_vpc.id
```

```
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
```

```

}

ingress {
  from_port = 80
  to_port   = 80
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
  prefix_list_ids = []
}

tags = {
  Name = "${var.env_prefix}-sg"
}
}

```

- **Save screenshot as:** task9_security_group_apply.png — main.tf showing security group resource (before apply).

```

GNU nano 7.2 main.tf
# Security Group
resource "aws_default_security_group" "myapp_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

```

Run apply:

terraform apply -auto-approve

- **Save screenshot as:** task9_security_group_apply.png — terminal showing terraform apply output for security group (after apply) and verify in AWS Console.

```

+ from_port      = 22
+ ipv6_cidr_blocks = []
+ prefix_list_ids = []
+ protocol       = "tcp"
+ security_groups = []
+ self           = false
+ to_port        = 22
# (1 unchanged attribute hidden)
},
]
+ name                = (known after apply)
+ name_prefix         = (known after apply)
+ owner_id            = (known after apply)
+ region              = "me-central-1"
+ revoke_rules_on_delete = false
+ tags                = {
+   "Name" = "dev-sg"
}
+ tags_all            = {
+   "Name" = "dev-sg"
}
+ vpc_id              = "vpc-01fbeb5b3c6ba1c3"
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_default_security_group.myapp_sg: Creating...
aws_default_security_group.myapp_sg: Creation complete after 3s [id=sg-05062e075ed12b71c]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ s|

```

4. Create an AWS key pair and save locally Create a key pair and store the private key in your Codespace. Do NOT commit the .pem file.

```

aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
  --query 'KeyMaterial' \
  --output text > MyED25519Key.pem

```

chmod 600 MyED25519Key.pem

- **Save screenshot as:** task9_keypair_created_and_saved.png — terminal showing key creation and MyED25519Key.pem saved.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
  --query 'KeyMaterial' \
  --output text > MyED25519Key.pem

chmod 600 MyED25519Key.pem
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```

Ensure .gitignore contains:

*.pem

- **Save screenshot as:** task9_keypair_created_and_saved.png — .gitignore showing *.pem entry.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
  --query 'KeyMaterial' \
  --output text > MyED25519Key.pem

chmod 600 MyED25519Key.pem
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```

5. Add EC2 instance resource (initial) Add the instance resource to main.tf (initially using the created key name):

```
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.default_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = "MyED25519Key"

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}
```

```
output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
```

- **Save screenshot as:** task9_instance_type_set.png — main.tf showing instance_type variable and aws_instance resource (before apply).

```
cidr_blocks = ["0.0.0.0/0"]
}

tags = {
  Name = "${var.env_prefix}-sg"
}
}

resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = [aws_default_security_group.myapp_sg.id]
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = "MyED25519Key"

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}
|
```

Run:

terraform apply -auto-approve

- **Save screenshot as:** task9_ec2_apply_and_public_ip.png — terraform apply output showing EC2 created and public IP (or terraform output).

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

Changes to Outputs:

- + aws_instance_public_ip = (known after apply)

```
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 12s [id=i-0a965a050fea29653]
```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

```
aws_instance_public_ip = "3.29.50.227"
```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) \$ |

- [illegible]

- Save screenshot as:** task9_ssh_keypair_and_ssh.png — terminal showing ssh-keygen output and files in ~/.ssh.
- Add a Terraform resource in main.tf to register the public key:
- ```
resource "aws_key_pair" "ssh_key" {
 key_name = "serverkey"
 public_key = file("~/ssh/id_ed25519.pub")
}
```
- Update the EC2 resource to use the Terraform-managed key:
- ```
resource "aws_instance" "myapp-server" {  
  ...  
  # replace key_name = "MyED25519Key" with:  
  key_name = aws_key_pair.ssh_key.key_name  
  ...  
}
```
- Save screenshot as:** task9_ssh_keypair_and_ssh.png — main.tf showing aws_key_pair resource and updated aws_instance key_name.
- Run:
- ```
terraform apply -auto-approve
```
- Save screenshot as:** task9\_ec2\_apply\_and\_public\_ip.png — terraform apply output

showing EC2 updated to use new key (and any changes).

```
aws_key_pair.ssh_key will be created
+ resource "aws_key_pair" "ssh_key" {
 + arn = (known after apply)
 + fingerprint = (known after apply)
 + id = (known after apply)
 + key_name = "serverkey"
 + key_name_prefix = (known after apply)
 + key_pair_id = (known after apply)
 + key_type = (known after apply)
 + public_key = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIERLHZKRZueveVULxYtLnCpggF2VVyRahMEQOuWBbx3l codesp
 + region = "me-central-1"
 + tags_all = (known after apply)
}

Plan: 2 to add, 0 to change, 1 to destroy.

Changes to Outputs:
 ~ aws_instance_public_ip = "3.29.50.227" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-0a965a050fea29653]
aws_instance.myapp-server: Still destroying... [id=i-0a965a050fea29653, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0a965a050fea29653, 00m20s elapsed]
aws_instance.myapp-server: Destruction complete after 30s
aws_key_pair.ssh_key: Creating...
aws_key_pair.ssh_key: Creation complete after 0s [id=serverkey]
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-0669f8b5fc59b5253]

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.

Outputs:

aws_instance_public_ip = "3.28.44.169"
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |
```

8. SSH using the newly registered key Now SSH with your generated private key (the default ssh client will pick up ~/.ssh/id\_ed25519):  
ssh ec2-user@<public-ip>  
Verify login works.
  - **Save screenshot as:** task9\_ssh\_keypair\_and\_ssh.png — terminal showing SSH session using the generated key.
  9. Install nginx via inline user\_data Modify the aws\_instance resource to include inline user\_data:  
resource "aws\_instance" "myapp-server" {  
 ...  
 user\_data = <<-EOF  
 #!/bin/bash  
 yum update -y  
 yum install -y nginx  
 systemctl start nginx  
 systemctl enable nginx  
 EOF  
 tags = {  
 Name = "\${var.env\_prefix}-ec2-instance"  
 }  
}
- Apply:  
terraform apply -auto-approve  
SSH in and run:  
curl localhost  
Open http://<public-ip> in a browser — you should see the nginx default page

- **Save screenshot as:** task9\_nginx\_local\_curl.png — curl localhost inside the EC2 instance showing nginx response.

```

~ public_ip = "3.28.44.169" -> (known after apply)
tags
 "Name" = "dev-ec2-instance"
}
+ user_data = <<-EOT
 #!/bin/bash
 yum update -y
 yum install -y nginx
 systemctl start nginx
 systemctl enable nginx
EOT
(38 unchanged attributes hidden)

(9 unchanged blocks hidden)
}

Plan: 0 to add, 1 to change, 0 to destroy.

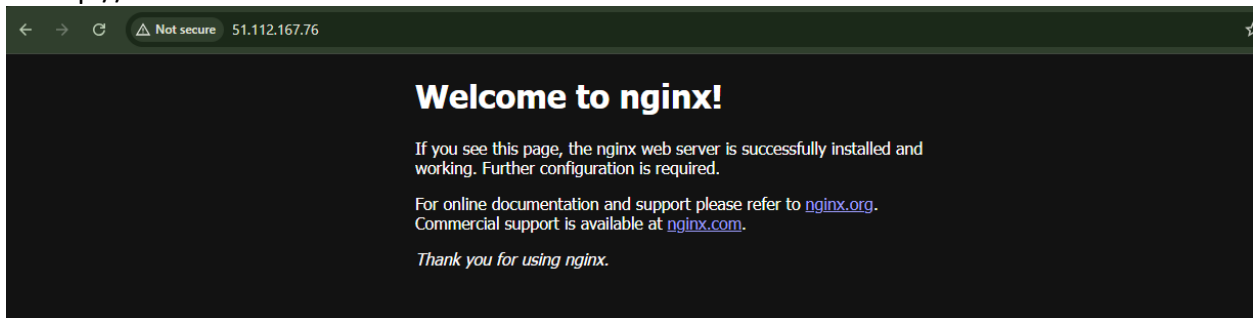
Changes to Outputs:
 ~ aws_instance_public_ip = "3.28.44.169" -> (known after apply)
aws_instance.myapp-server: Modifying... [id=i-0669f8b5fc59b5253]
aws_instance.myapp-server: Still modifying... [id=i-0669f8b5fc59b5253, 00m10s elapsed]
aws_instance.myapp-server: Still modifying... [id=i-0669f8b5fc59b5253, 00m20s elapsed]
aws_instance.myapp-server: Still modifying... [id=i-0669f8b5fc59b5253, 00m30s elapsed]
aws_instance.myapp-server: Still modifying... [id=i-0669f8b5fc59b5253, 00m40s elapsed]
aws_instance.myapp-server: Still modifying... [id=i-0669f8b5fc59b5253, 00m50s elapsed]
aws_instance.myapp-server: Still modifying... [id=i-0669f8b5fc59b5253, 01m00s elapsed]
aws_instance.myapp-server: Modifications complete after 1m5s [id=i-0669f8b5fc59b5253]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:
aws_instance_public_ip = "51.112.52.91"
@hamail-maker ~$ cd /workspaces/CF-hamail-maker-023-Lab11 (main) $ |

```

- **Save screenshot as:** task9\_nginx\_browser\_page.png — browser showing nginx default page at http://.



10. Use an external script for user\_data Create a script file in the Codespace:

```

cat > entry-script.sh <<'EOF'
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
EOF

```

- **Save screenshot as:** task9\_nginx\_local\_curl.png — entry-script.sh created and permissions set.

```
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ cd /workspaces/CC_hamail-maker_023_Lab11

cat > entry-script.sh <<'EOF'
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
EOF
```

Update aws\_instance to use the file:

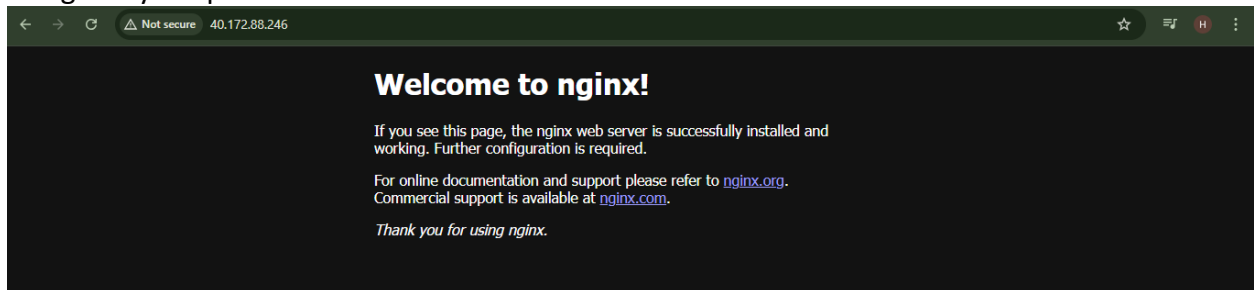
```
resource "aws_instance" "myapp-server" {
 ...
 user_data = file("entry-script.sh")
 tags = {
 Name = "${var.env_prefix}-ec2-instance"
 }
}
```

Apply:

terraform apply -auto-approve

Reload the browser at <http://<public-ip>> to confirm nginx is served.

- **Save screenshot as:** task9\_nginx\_browser\_page.png — browser showing nginx page after using entry-script.sh.



## Cleanup

1. Destroy all resources:  
terraform destroy -auto-approve
- **Save screenshot as:** cleanup\_destroy.png — terraform destroy output showing resources deleted.

```
Destroy complete! Resources: 7 destroyed.
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ s|
```

2. Verify state files:  
cat terraform.tfstate  
cat terraform.tfstate.backup
- **Save screenshot as:** cleanup\_state\_files.png — showing terraform.tfstate and terraform.tfstate.backup contents (ensure no secrets are exposed).

```

Destroy complete! Resources: 7 destroyed.
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ ls -l terraform.tfstate terraform.tfstate.backup
-rw-rw-rw- 1 codespace codespace 182 Dec 20 14:53 terraform.tfstate
-rw-rw-rw- 1 codespace codespace 15660 Dec 20 14:52 terraform.tfstate.backup
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ cat terraform.tfstate
{
 "version": 4,
 "terraform_version": "1.14.3",
 "serial": 57,
 "lineage": "54fc27f1-d672-853b-208d-a22b55ebce73",
 "outputs": {},
 "resources": [],
 "check_results": null
}
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ cat terraform.tfstate.backup
{
 "version": 4,
 "terraform_version": "1.14.3",
 "serial": 49,
 "lineage": "54fc27f1-d672-853b-208d-a22b55ebce73",
 "outputs": {}
}

```

3. Ensure no sensitive files are committed. If any private keys or credentials were accidentally committed — rotate them immediately.
- **Save screenshot as:** cleanup\_verify\_no\_secrets.png — git status and .gitignore confirming no .pem, .tfstate, or credentials are staged.

```

@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
 (use "git add <file>..." to include in what will be committed)
 .gitignore
 .terraform.lock.hcl
 aws/
 awscli2.zip
 entry-script.sh
 id_ed25519
 id_ed25519.pub
 locals.tf
 main.tf
 main.tf.save
 main.tf.save.1

nothing added to commit but untracked files present (use "git add" to track)
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ cat .gitignore
.terraform/*
*.tfstate
.tfstate.
*.tfvars
*.pem
@Hamail-maker → /workspaces/CC_hamail-maker_023_Lab11 (main) $ |

```